

Билет 7. Системный вызов `futex`.

Реализация `std::mutex` через `futex`. Идея реализации `futex` в ядре ОС

0. Формулировка билета

Системный вызов `futex`. Реализация `std::mutex` через `futex`. Идея реализации `futex` в ядре ОС.

Главная идея билета:

Спинлок хорош для очень короткого ожидания, но если критическая секция долгая, активное ожидание тратит CPU впустую.

`futex` позволяет сделать быстрый путь мьютекса в userspace через атомики, а при конкуренции усыпить поток в ядре.

Ключевая схема:

```
uncontended lock:
  atomic exchange
  no syscall

contended lock:
  atomic failed
  futex_wait
  thread sleeps in kernel

unlock:
  store unlocked
  futex_wake one waiter
```

1. Зачем нужен `futex`

В прошлом билете был spinlock:

```
while (state.exchange(true)) {
    // spin
}
```

Если lock занят, поток активно крутится.

Это нормально, когда:

критическая секция очень короткая
lock скоро освободится
потоков не больше, чем ядер

Но плохо, если:

критическая секция долгая
внутри I/O
поток-владелец вытеснен с CPU
много потоков конкурируют за lock

Тогда ждущие потоки бесполезно занимают CPU.

Нужен механизм:

если lock занят надолго:
не крутиться
заснуть
проснуться, когда lock освободят

Обычный `std::mutex` примерно так и работает:

быстрый путь:
атомарно захватить в userspace

медленный путь:
если не получилось — уйти в ядро и уснуть

Для этого в Linux есть `futex`.

2. Что такое `futex`

`futex` = **fast userspace mutex**.

Идея:

Состояние синхронизационного примитива хранится в обычной памяти пользователя, обычно в 32-битном слове.
Пока нет конкуренции, ядро вообще не участвует.
Ядро нужно только тогда, когда поток должен уснуть или разбудить других.

То есть `futex` — это не полноценный объект-мьютекс в ядре.

`futex` — это системный вызов, который умеет:

усыпить поток, если по адресу лежит ожидаемое значение
разбудить потоки, ожидающие на этом адресе

Главные операции:

```
FUTEX_WAIT(addr, expected)
FUTEX_WAKE(addr, n)
```

3. FUTEX_WAIT

Операция:

```
FUTEX_WAIT(addr, expected)
```

означает:

```
посмотри на 32-битное слово по адресу addr

если *addr == expected:
    поставь поток в очередь ожидания этого адреса
    усыпи поток

если *addr != expected:
    сразу вернись, не засыпая
```

Псевдокод:

```
futex_wait(&state, 1);
```

Смысл:

```
усни только если state всё ещё равно 1
```

Для мьютекса:

```
state == 0 → свободен
state == 1 → занят
```

Если поток увидел, что мьютекс занят, он вызывает:

```
futex_wait(&state, 1);
```

То есть:

```
если мьютекс всё ещё занят — спать  
если он уже освободился — не спать, вернуться
```

4. Почему `FUTEX_WAIT` проверяет значение

Это ключевой момент.

Плохой сценарий без проверки значения:

```
thread A:  
    увидел state == 1  
    решил спать  
  
thread B:  
    unlock  
    state = 0  
    wake  
  
thread A:  
    только теперь реально заснул  
  
итог:  
    wake потерялся  
    А спит навсегда
```

Это называется **lost wakeup** — потерянное пробуждение.

`futex_wait(addr, expected)` решает эту проблему, потому что проверка значения и постановка потока в очередь ожидания происходят атомарно внутри ядра.

То есть поток не делает руками:

```
проверил  
отпустил CPU  
заснул
```

Он делает:

ядро проверило значение и либо усыпило, либо сразу вернуло

Если значение уже изменилось, поток не заснёт.

5. FUTEX_WAKE

Операция:

```
FUTEX_WAKE(addr, n)
```

означает:

разбудить до n потоков,
которые ждут на futex-адресе addr

Пример:

```
futex_wake(&state, 1);
```

Смысл:

разбудить одного потока,
который ждёт изменения state

Для mutex обычно будят одного:

освободился один mutex
продолжить может один поток

Если будить всех, получится лишняя конкуренция:

много потоков проснулись
один захватил mutex
остальные снова уснули

Это похоже на `notify_one` и `notify_all` у `condition_variable`.

6. Важное отличие `futex` от `condition_variable`

`futex` ждёт значение **одной ячейки памяти**.

ждать, пока `state` перестанет быть 1

`condition_variable` ждёт произвольный предикат на произвольном состоянии.

ждать, пока `!queue.empty()`
ждать, пока `stop || !tasks.empty()`
ждать, пока `count > 0`

Поэтому:

`futex` – низкоуровневый строительный блок
`condition_variable` – более высокоуровневый примитив

На `futex` можно строить:

`mutex`
`semaphore`
`condition_variable`
`barrier`
`blocking queue`

Но наружу пользователю обычно дают более удобный примитив.

7. Сигнатура `futex` в Linux

Реальный системный вызов выглядит сложнее:

```
int futex(  
    int* uaddr,  
    int futex_op,  
    int val,  
    const struct timespec* timeout,  
    int* uaddr2,  
    int val3  
);
```

Для базового понимания достаточно двух операций:

```
futex(uaddr, FUTEX_WAIT, expected, ...);  
futex(uaddr, FUTEX_WAKE, n, ...);
```

На практике часто используют:

```
FUTEX_WAIT_PRIVATE  
FUTEX_WAKE_PRIVATE
```

если futex используется только внутри одного процесса.

`PRIVATE`-варианты могут быть быстрее, потому что ядру не нужно учитывать межпроцессное разделение памяти.

8. User space fast path

Главная сила futex — быстрый путь без системного вызова.

Для мьютекса:

```
если свободен:  
    atomic exchange захватил lock  
    всё произошло в userspace  
    syscall не нужен
```

Пример:

```
if (state.exchange(1) == 0) {  
    return;  
}
```

Если `state` был `0`, мы записали `1` и захватили мьютекс.

Это быстрый путь.

```
uncontended mutex:  
    один atomic RMW  
    no kernel  
    no context switch
```

Системный вызов нужен только при конкуренции:

```
если state уже был 1:  
    кто-то держит mutex  
    можно идти в futex_wait
```

9. Почему это быстрее обычного “kernel mutex”

Если бы каждый `lock` / `unlock` всегда ходил в ядро, даже когда конкуренции нет, это было бы дорого.

Системный вызов — это:

```
переход user → kernel  
проверки  
работа планировщика  
возможное переключение контекста  
возврат kernel → user
```

`futex` даёт идею:

```
нет конкуренции → ядро не участвует  
есть конкуренция → ядро паркует поток
```

Поэтому современные мьютексы обычно устроены примерно так:

```
atomic fast path  
+  
futex slow path
```

10. Двухсостояний mutex через futex

Состояния:

```
0 — свободен  
1 — занят
```

Код:

```
struct Mutex {  
    std::atomic<int> state{0};  
};
```



```

void lock() {
    if (state.exchange(1, std::memory_order_acquire) == 0) {
        return;
    }

    do {
        futex_wait(&state, 1);
    } while (state.exchange(1, std::memory_order_acquire) != 0);
}

void unlock() {
    state.store(0, std::memory_order_release);
    futex_wake(&state, 1);
}
};

```

Это учебная реализация.

Она показывает главную идею:

```

lock:
    пробуем захватить атомарно
    если занято – спим через futex_wait
    после пробуждения снова пробуем захватить

unlock:
    помечаем свободным
    будим одного ожидающего

```

11. Почему в `lock` нужен цикл

`futex_wait` может вернуться по разным причинам:

```

кто-то вызвал futex_wake
пришёл сигнал
значение не совпало с expected
spurious wakeup / ложное пробуждение на уровне логики примитива
несколько потоков проснулись, а lock забрал только один

```

Поэтому нельзя написать:

```

futex_wait(&state, 1);
state.exchange(1);

```

Нужно:

```
do {  
    futex_wait(&state, 1);  
} while (state.exchange(1) != 0);
```

То есть после каждого пробуждения снова пытаемся захватить lock.

Аналогия с `condition_variable`:

```
while (!predicate) {  
    cv.wait(lock);  
}
```

12. Почему `unlock` сначала `store(0)`, потом `wake`

Правильный порядок в базовой схеме:

```
state.store(0);  
futex_wake(&state, 1);
```

Почему?

Потому что ожидающий поток должен проснуться и увидеть, что состояние изменилось.

Если переставить:

```
futex_wake(&state, 1);  
state.store(0);
```

может случиться lost wakeup.

Сценарий:

```
thread A:  
    собирается заснуть, потому что state == 1  
  
thread B:  
    wake, но ещё state == 1  
    wake никого полезно не разбудил  
  
thread B:  
    state = 0  
  
thread A:
```

```
засыпает с ожиданием state == 1  
  
будить уже некому
```

Поэтому:

```
сначала меняем состояние  
потом будим
```

Так же как с `condition_variable`:

```
сначала меняем predicate  
потом notify
```

13. Но двухсостояние реализация имеет проблему

Учебная реализация:

```
void unlock() {  
    state.store(0);  
    futex_wake(&state, 1);  
}
```

имеет тонкую проблему.

После:

```
state.store(0);
```

мьютекс уже считается свободным.

Другой поток может:

```
захватить этот мьютекс  
закончить работу с объектом  
уничтожить объект, внутри которого лежит мьютекс
```

А первый поток ещё не дошёл до:

```
futex_wake(&state, 1);
```

Он может вызвать `futex_wake` по адресу, который уже относится к уничтоженному объекту.

Это use-after-free.

Подробно это будет в следующем билете про мьютекс на трёх состояниях и `OneShot`.

Для билета 7 достаточно сказать:

Простая двухсостояниея реализация показывает идею futex-мьютекса, но не является полностью корректной промышленной реализацией. У неё есть проблема между `store(0)` и `futex_wake`, поэтому дальше вводят мьютекс на трёх состояниях.

14. Спинлок vs futex-мьютекс

Свойство	Spinlock	Futex-mutex
Если lock занят	поток крутится	поток может уснуть
CPU при ожидании	тратится	почти не тратится
Быстрый путь	атомик	атомик
Медленный путь	всё ещё spin	syscall + parking
Хорош для	очень коротких секций	более долгих секций
Плохо при	долгом ожидании	частой конкуренции из-за syscalls
Context switch	нет	возможен

Главное:

spinlock минимизирует latency при коротком ожидании
futex-mutex экономит CPU при долгом ожидании

15. Adaptive mutex

На практике часто используют гибрид:

сначала немного покрутиться
если lock быстро освободился — захватить без сна
если ожидание затянулось — уйти в `futex_wait`

Это называется adaptive mutex.

Почему это полезно?

Если владелец lock-а прямо сейчас работает на другом ядре и скоро освободит lock, дешевле чуть-чуть покрутиться, чем делать syscall и context switch.

Но если ожидание долгое, надо спать.

Схема:

```
try lock
if failed:
    spin for a while
if still failed:
    futex_wait
```

16. Идея реализации futex в ядре ОС

В userspace есть адрес:

```
std::atomic<int> state;
```

Поток вызывает:

```
futex_wait(&state, 1);
```

Ядро не хранит заранее отдельный объект для каждого futex.

Вместо этого адрес используется как ключ.

Упрощённо внутри ядра:

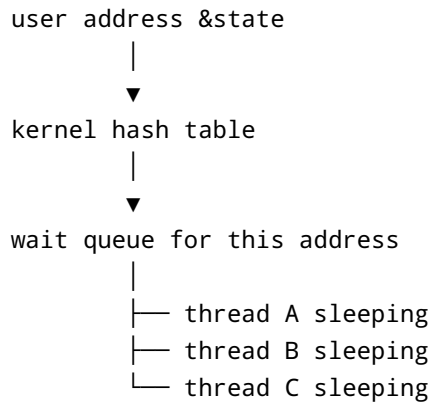
```
key = address of futex word
bucket = hash(key)

FUTEX_WAIT:
    проверить, что *addr == expected
    если нет — вернуть EAGAIN
    если да:
        добавить текущий поток в очередь bucket/key
        перевести поток в состояние sleep
        вызвать scheduler

FUTEX_WAKE:
    найти bucket/key по addr
    взять до n потоков из очереди ожидания
```

пометить их runnable
scheduler потом даст им CPU

Схема:



То есть futex-адрес — это не указатель на kernel object.

Это ключ, по которому ядро находит очередь ожидающих потоков.

17. Почему futex считается fast userspace mutex

Потому что ядро участвует только на медленном пути.

```
lock свободен:
    atomic exchange
    return
    no syscall

lock занят:
    futex_wait
    thread sleeps

unlock:
    store unlocked
    futex_wake if needed
```

В идеале:

частый случай — uncontended
редкий случай — contended

И тогда средняя цена мьютекса низкая.

18. EAGAIN, EINTR, spurious wakeup

`futex_wait` может вернуться не только потому, что lock освободили.

Возможные причины:

значение по адресу уже не равно expected → EAGAIN
системный вызов прерван сигналом → EINTR
поток разбудили, но lock забрал кто-то другой

Поэтому после `futex_wait` всегда надо перепроверять состояние.

Для mutex:

```
while (state.exchange(1) != 0) {  
    futex_wait(&state, 1);  
}
```

Для condition variable:

```
while (!predicate()) {  
    cv.wait(lock);  
}
```

Это одна и та же философия:

проснулся → не верь пробуждению → проверь состояние

19. Memory order в futex-мьютексе

Для lock/unlock нужны гарантии видимости данных критической секции.

На захвате:

```
state.exchange(1, std::memory_order_acquire)
```

На освобождении:

```
state.store(0, std::memory_order_release)
```

Смысл:

```
release на unlock:
    записи внутри critical section не переедут после unlock

acquire на lock:
    следующий владелец увидит записи предыдущего владельца
```

Сам `futex_wait` и `futex_wake` — это механизм сна/пробуждения.

А атомики вокруг state отвечают за состояние lock-а и memory ordering.

20. Почему futex не заменяет атомики

Нельзя построить mutex только на `futex_wait/wake`.

Нужны атомики для быстрого пути и для изменения состояния:

```
futex_wait:
    только спит, если значение равно expected

атомик:
    проверяет и меняет состояние lock-а
```

Для lock нужна операция:

```
если state == 0:
    сделать state = 1
```

Это RMW-операция, например `exchange` или CAS.

`futex` сам по себе не делает “захват mutex-а”.

Он только помогает ждать.

21. Короткий ответ на экзамене

Можно отвечать так:

`futex` — это системный вызов Linux, который позволяет строить быстрые синхронизационные примитивы. Состояние примитива хранится в userspace, обычно в 32-битном слове. Пока нет конкуренции, мьютекс захватывается атомарной операцией без системного вызова. Если мьютекс занят, поток вызывает `FUTEX_WAIT(addr, expected)`.

`FUTEX_WAIT` проверяет, что по адресу всё ещё лежит `expected`, и если да, атомарно ставит поток в очередь ожидания и усыпляет. Если значение уже изменилось, поток не засыпает и вызов возвращается сразу. Это защищает от lost wakeup. `FUTEX_WAKE(addr, n)` будит до `n` потоков, ожидающих на этом адресе.

Простая реализация mutex-а: `state = 0` означает свободен, `state = 1` означает занят. В `lock` делаем `state.exchange(1)`: если старое значение было 0, захватили без syscall. Если было 1, вызываем `futex_wait(&state, 1)` и после пробуждения снова пробуем захватить. В `unlock` делаем `state.store(0)` и `futex_wake(&state, 1)`.

В ядре futex реализуется как очередь ожидания, привязанная к адресу пользовательской памяти: адрес хешируется, по нему находится bucket с ожидающими потоками. `WAIT` добавляет поток в эту очередь и паркует его, `WAKE` достаёт одного или несколько потоков и делает их runnable.

Простая двухсостояниеовая реализация показывает идею, но имеет тонкие проблемы: лишние wake-ы и use-after-free между `store(0)` и `futex_wake`. Поэтому в следующем шаге вводят мьютекс на трёх состояниях.

22. Мини-код для ответа на доске

```
struct Mutex {
    std::atomic<int> state{0}; // 0 – free, 1 – locked

    void lock() {
        if (state.exchange(1, std::memory_order_acquire) == 0) {
            return; // fast path
        }

        while (true) {
            futex_wait(&state, 1);

            if (state.exchange(1, std::memory_order_acquire) == 0) {
                return;
            }
        }
    }

    void unlock() {
        state.store(0, std::memory_order_release);
        futex_wake(&state, 1);
    }
};
```

Что объяснить:

state хранится в userspace
exchange – быстрый путь без syscall
futex_wait – усыпляет только если state всё ещё 1
futex_wake – будит одного ожидающего
после wait нужен цикл и повторная попытка захвата
store(0) должен идти перед wake
двухсостояниеверсия учебная, не идеальная

23. Мини-проверка

1. Зачем нужен `futex`, если уже есть `spinlock`?
2. Что означает название `fast userspace mutex`?
3. Что делает `FUTEX_WAIT(addr, expected)`?
4. Что делает `FUTEX_WAKE(addr, n)`?
5. Почему `FUTEX_WAIT` проверяет значение перед сном?
6. Почему `futex` помогает избежать `lost wakeup`?
7. Как устроен быстрый путь `mutex`-а на `futex`?
8. Как устроен медленный путь `mutex`-а на `futex`?
9. Почему после `futex_wait` нужно снова проверять состояние?
10. Почему `unlock` делает `store(0)`, а потом `futex_wake`?
11. Как примерно ядро ОС реализует `futex`?
12. Чем `futex-mutex` отличается от `spinlock`?
13. Почему простая двухсостояниеверсия реализация не является идеальной?
14. Что такое `adaptive mutex`?
15. Почему `futex` не заменяет атомики?